

Installation and Setup Instructions

Solari — Solar AI Chatbot (Track 2)

Prepared by:	Zain	Intern No.:	95
Task:	Installation and Setup Instructions	Date:	August 17, 2026
Project:	Solari — Solar AI Chatbot	Source Package:	Solar-AI-Chatbot-main.zip

1. Purpose

This document provides step-by-step instructions to install, configure, and run the Solari solar consultation chatbot prototype on a local machine. Solari consists of a Python FastAPI backend (AI integration, knowledge base, deterministic sizing engine) and an HTML/CSS/JavaScript frontend, connected through a REST API and powered by Google Gemini.

2. Prerequisites

Before installation, ensure the following are available on the machine:

- **Python 3.10 or newer** installed and available on the system PATH.
- A **Google Gemini API key** (required for the AI chat functionality).
- A text editor or IDE (e.g. VS Code) for editing configuration files.
- The project source package (`Solar-AI-Chatbot-main.zip`) extracted to a local folder.
- (Optional but recommended) A local static web server to serve the frontend, instead of opening files directly via `file://`.

3. Project Structure Overview

The project is split into two main layers:

- **backend/** — FastAPI application, Gemini orchestration, knowledge base, sizing engine, environment configuration.
- **frontend/** — HTML, CSS, and JavaScript single-page interface (chat UI, knowledge base viewer, calculator).

4. Backend Setup

Step 1 — Open a terminal in the backend directory

```
cd Solar-AI-Chatbot-main/backend
```

Step 2 — Create a virtual environment

```
python -m venv .venv
```

Step 3 — Activate the virtual environment

```
# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate
```

Step 4 — Install dependencies

```
pip install -r requirements.txt
```

Step 5 — Configure environment variables

Create a `.env` file inside the `backend/` directory (do not commit this file to source control) and add the following:

```
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-3.1-flash-lite # optional, if not using the default
```

Important: Do not reuse or commit any credential found in `.env.example`. If a real key is present there, revoke and rotate it before use.

Step 6 — Start the backend server

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Once running, verify the backend is healthy by visiting:

```
http://127.0.0.1:8000/health
http://127.0.0.1:8000/
```

Windows Helper Script (optional)

On Windows, `backend/run.bat` automates virtual environment creation, activation, dependency installation, and server startup. Simply double-click or run it from the terminal:

```
run.bat
```

5. Frontend Setup

The frontend is a static HTML/CSS/JavaScript application located in the `frontend/` directory. It communicates with the backend via `127.0.0.1:8000` during local development.

Step 1 — Serve the frontend using a local web server

Opening `index.html` directly (file://) can cause issues with API requests. It is recommended to serve it locally, for example:

```
cd Solar-AI-Chatbot-main/frontend
python -m http.server 5500
```

Then open `http://127.0.0.1:5500` in a browser.

Step 2 — Confirm the connection

With the backend running on port 8000 and the frontend served locally, open the chat interface and send a test message (e.g. a general solar question) to confirm the frontend successfully reaches the `/api/chat` endpoint.

6. Verifying the Installation

- **Health check:** GET `/health` should return a successful service status.
- **Chat test:** Send a message through the chat UI and confirm a grounded AI response is returned.
- **Sizing test:** Use the system calculator (monthly units + backup hours) and confirm POST `/api/recommend` returns a deterministic sizing result.
- **Knowledge base:** Confirm the Knowledge Base tab displays the three reference PDFs.
- **Test cases:** Confirm the Test Cases tab exposes the Excel test workbook (30 test cases).

7. Troubleshooting

Issue	Likely Cause / Fix
Server fails to start	Confirm Python 3.10+ is active in the virtual environment and dependencies installed correctly.
Chat returns errors / no AI response	Confirm GEMINI_API_KEY is set correctly in backend/.env and has not been rate-limited or exhausted.
Frontend cannot reach backend	Confirm the backend is running on port 8000 and CORS is enabled; check the API base URL in frontend/js/script.js.
"Module not found" errors	Re-run <code>pip install -r requirements.txt</code> inside the activated virtual environment.
Sizing values look off	Sizing uses fixed engineering constants (peak sun hours, panel wattage, etc.); this is an estimate, not a site survey.

8. Notes for Deployment (Beyond Local Setup)

The following changes are required if the prototype is deployed beyond local development (not needed for local installation, but useful for graders/reviewers to know):

- Replace hard-coded `127.0.0.1:8000` API URLs in the frontend with the deployed backend URL.
- Restrict CORS (`allow_origins`) to the actual deployed frontend origin instead of `'*'`.
- Run the server without `--reload` in production.
- Serve the frontend via proper static hosting and use HTTPS.

- Keep the Gemini API credential outside the repository and rotate any credential that was ever exposed.